REACTIVEDYNAMICS.JL — THE AGENTIC LAYER

# ReactiveDynamics.jl Modeling Copilot

Describe the operation in conversation. The copilot builds a high-fidelity model beneath it, one that carries the shared capacity, lead times, feedback and mid-course decisions a spreadsheet flattens, tries to break it, and uses it to compare the options and explain what drives the answer. Nobody writes or edits the model by hand.

THE PROBLEM

# Decisions about how an operation behaves need a model, and a model needs a modeler

- A launch reservation, a bid under a staffing ceiling, a pipeline whose funding follows its results, a backlog that feeds back into demand: the answer depends on **how the operation behaves over time**.
- A workbook can hold a time series and even a Monte Carlo draw. It cannot easily hold one shared line allocated month by month between products with random launch dates, plus a rule that changes the plan when a readout lands. That takes a simulator nobody can check.
- So these questions go to a specialist, take weeks, and come back as a workbook the decision-maker has to take on faith.
- Most never get a model at all.

# More decisions get a real model, the owner stays in the loop, the answer travels

- **A model for the cost of a conversation.** The threshold for a real model drops from a specialist's weeks to a guided session with the decision owner.
- **The owner stays in the loop.** The interview runs in business language with the person who owns the decision. Every assumption lands in a brief they signed, including which inputs were their own estimates.
- **The answer travels.** A study ends in a generated report the modeler can forward: the answer, the model, the evidence and the provenance, laid out to one template.
- **It compounds.** The next question about the same operation starts from the same model.

THE REACH

# What it can model — and what the copilot checks today

| Pattern | Example | Copilot today |
|---|---|---|
| **Contention** | two launches on one fill line | authored and checked end to end |
| **Lead times, clocks** | tech transfer, a build, a review | authored and checked |
| **Feedback** | backlog throttles intake; share lost after a stock-out | authored and run (12 state-dependent rates in the case); loop checks next (W25) |
| **Mid-course decisions** | review a contract at a readout | authored and run; the case turned on one |
| **Lifecycles** | projects moving through phases | authored and run |
| **Coupled models** | supply feeding commercial; several sites | in the engine (AlgebraicAgents); not yet through the copilot (W25) |

HOW IT WORKS

# Four steps, and a check that can stop the answer

## From a question to a number you can take to a board

Four steps, and the third one is allowed to stop the other three.

**01**

### Describe the decision

A conversation: the choice, what drives it, how the parts interact. If arithmetic answers it, we stop here.

**02**

### We build your operation

A running model of how it behaves — shared capacity, delays, feedback, decisions taken mid-course, cash.

**03**

### We try to break it

Nine checks ask whether the model does what you said the business does. A failure blocks the answer.

**04**

### A number you can defend

A central estimate with the evidence behind it — and a seed that reproduces it exactly.

**If the model does not behave like your business, you get the diagnosis, not a number.**

It names the check that failed and what it means: fix the model, or revisit the brief.

# The model is data, not code, so it can be checked before it runs

**Static checks.** A pure validator reads the model document and returns every problem with its exact location. A malformed model produces diagnostics, not a plausible wrong number. The language model proposes, a deterministic checker disposes, and the loop repeats until the checker is silent.

**The model is never code.** It is a closed vocabulary the engine compiles; nothing in a document can name a program.

**Where it stops today.** In the case, the odds-weighting behind the headline ran in session-written scripts: published, re-run independently, but reviewed by reading. Moving it into `rdx effect` is W27.

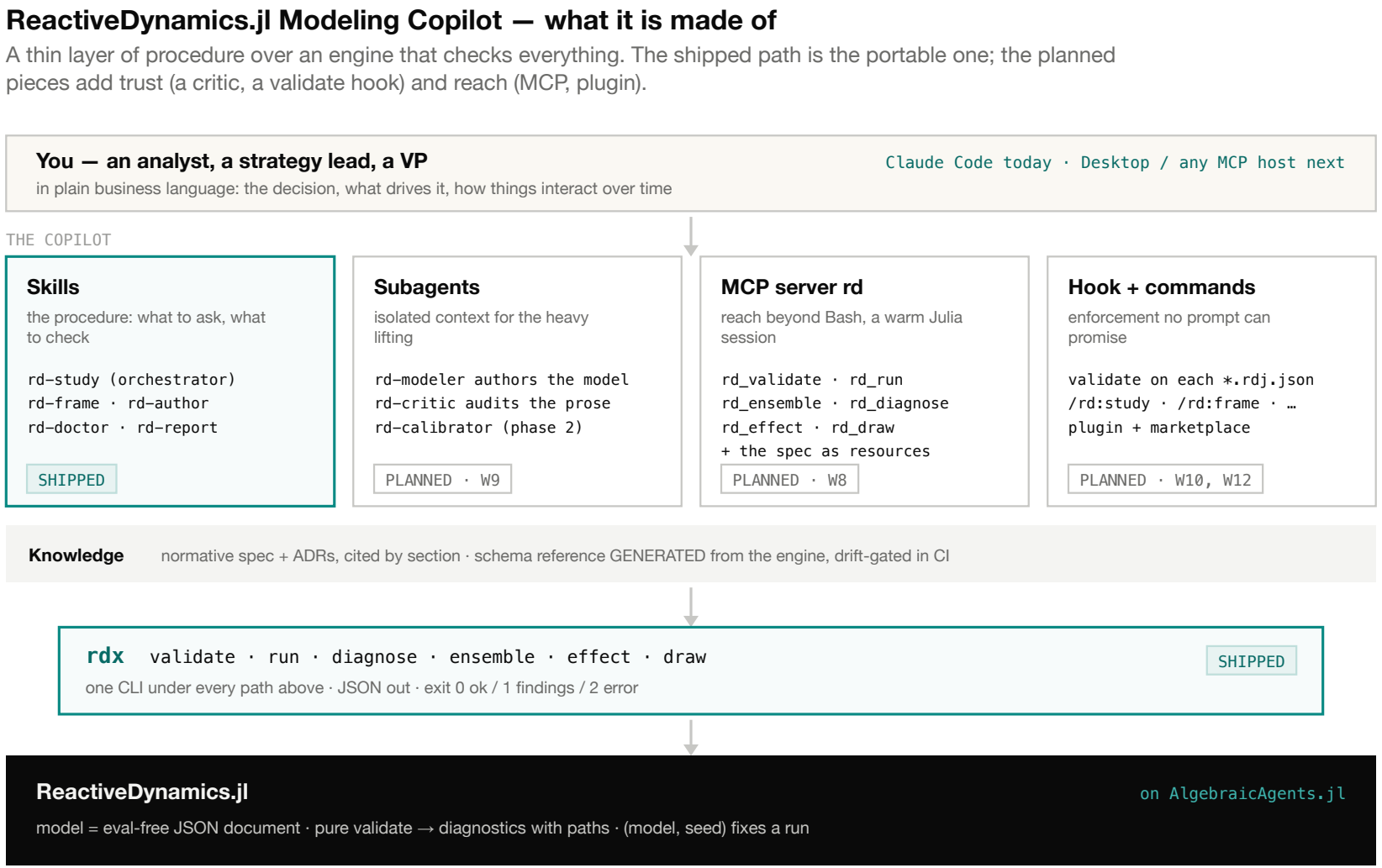
**The model document.** Places that hold resources, transitions that hold them over time and sometimes fail, rates that read the live state, rules that decide mid-course, costs, what each option changes. Diffable, and replayable from the model and a seed.

**Checked again, for meaning.** Plausibility checks ask whether the model does what the business does: a bottleneck that never binds, a step that never fires, a horizon that cuts off the payoff.

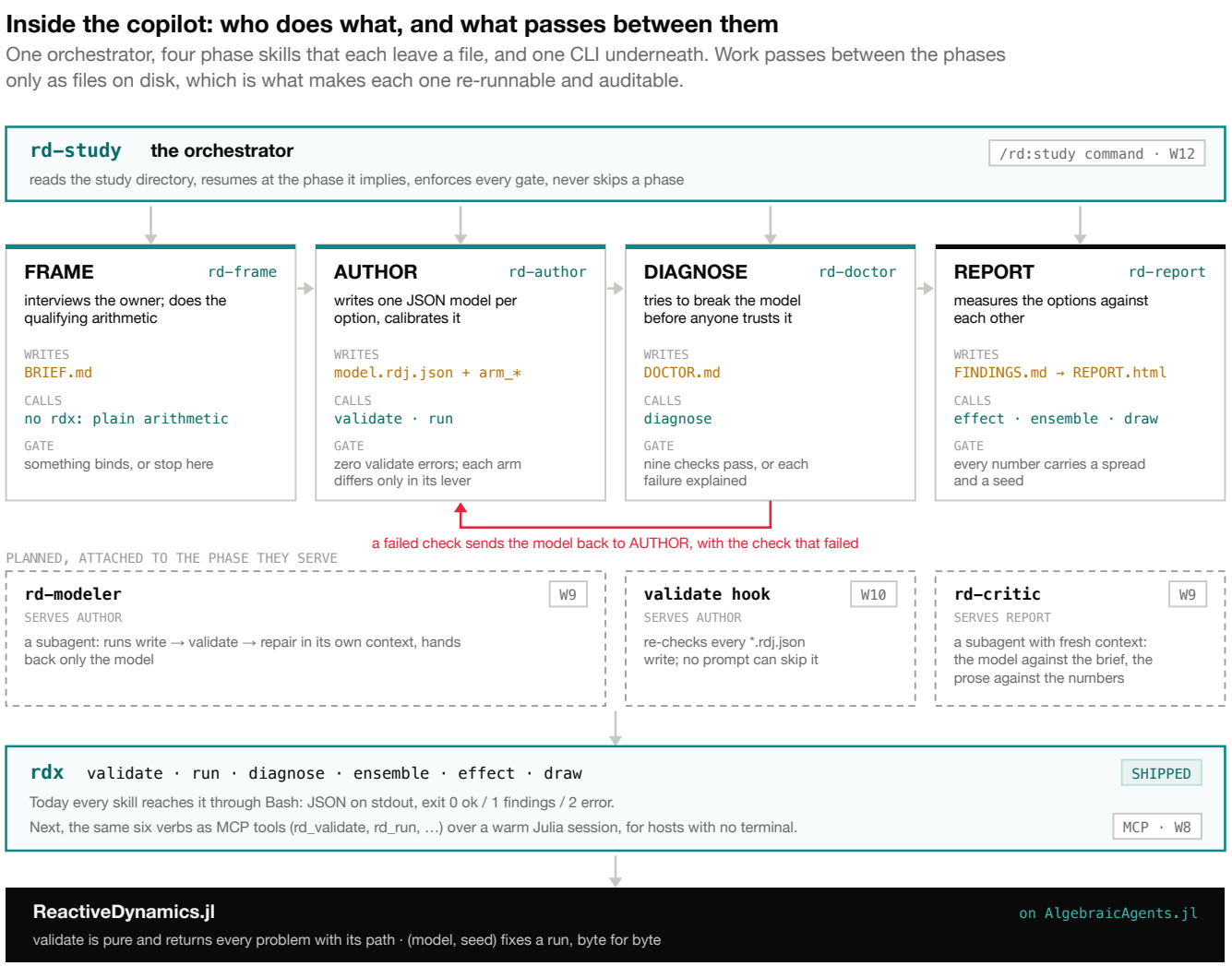
**Guarded in CI.** The schema reference is generated from the engine, and a known-answer test re-derives a published result on every change.

WHAT IT IS MADE OF

# A thin layer over an engine that checks everything



# Who does what, and what passes between them



A CAPTURED SESSION

# Launch supply: do we pay for capacity before the readouts?

Both sides were AI: the operator was a role-played agent answering from a fact sheet sealed before the first message; the assistant never saw it. The company is invented; the numbers are real runs of the model.

# Two Phase-3 biologics, one fill-finish line, a CMO offer that expires 30 November

**Kestrel** reads out first, at 60% odds; **Tern** nine months later, at 45%. Both would need the same line.

OPERATOR

Two Phase-3 biologics are coming. […] We have one sterile fill-finish line in-house, and most of it is already taken by our marketed products. It can probably carry one launch but not two. […] Our CFO says we shouldn't pay for capacity on an asset that has less than even odds.

CLAUDE

**The CMO slot works for Tern's filing, but only just.** If you sign by 30 November, PPQ is done somewhere between late 2027 and early 2028. […] So the CMO can be Tern's filed site with no PAS, but only if the CMO runs on time and Tern doesn't read out early.

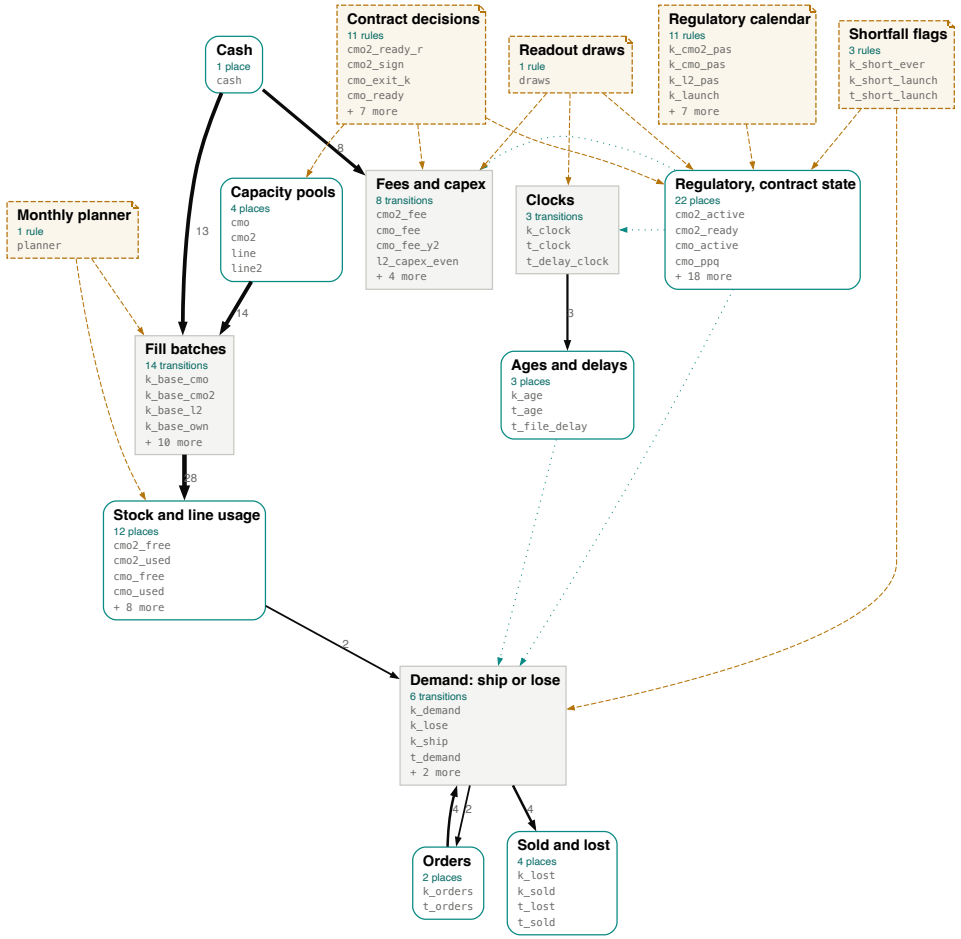
```
python3 -c '<in-house headroom: 12.8 batches/mo minus marketed 9/mo growing 5%/yr>'
```

# One model, a version per option, checked against the operator's own figures

- A generator script writes one network; the options differ only in parameters.
- `rdx validate`: 0 errors on every document.
- Consistency check: Kestrel first-year revenue **$250.0M** against the operator's ~$250M, and **54** batches a year at peak. Both follow from the inputs by construction; they show the inputs went in right.
- `rdx diagnose`: the own line binds. Every option fails one check by design: one network holds every option and readout outcome, so a single run leaves the other paths' steps idle. `DOCTOR.md` shows each fires where its path is taken, before any number is reported.
- Options compared on **paired paths**: the same seed gives every option the same readout dates and lead times.
- Readout outcomes are forced per path and weighted by the stated odds (0.27 / 0.33 / 0.18 / 0.22).
- No option-level ensembles were run: a 96-seed ensemble took 53 minutes (W24). One wait ensemble served as a cross-check, and replays of three of its members matched exactly.

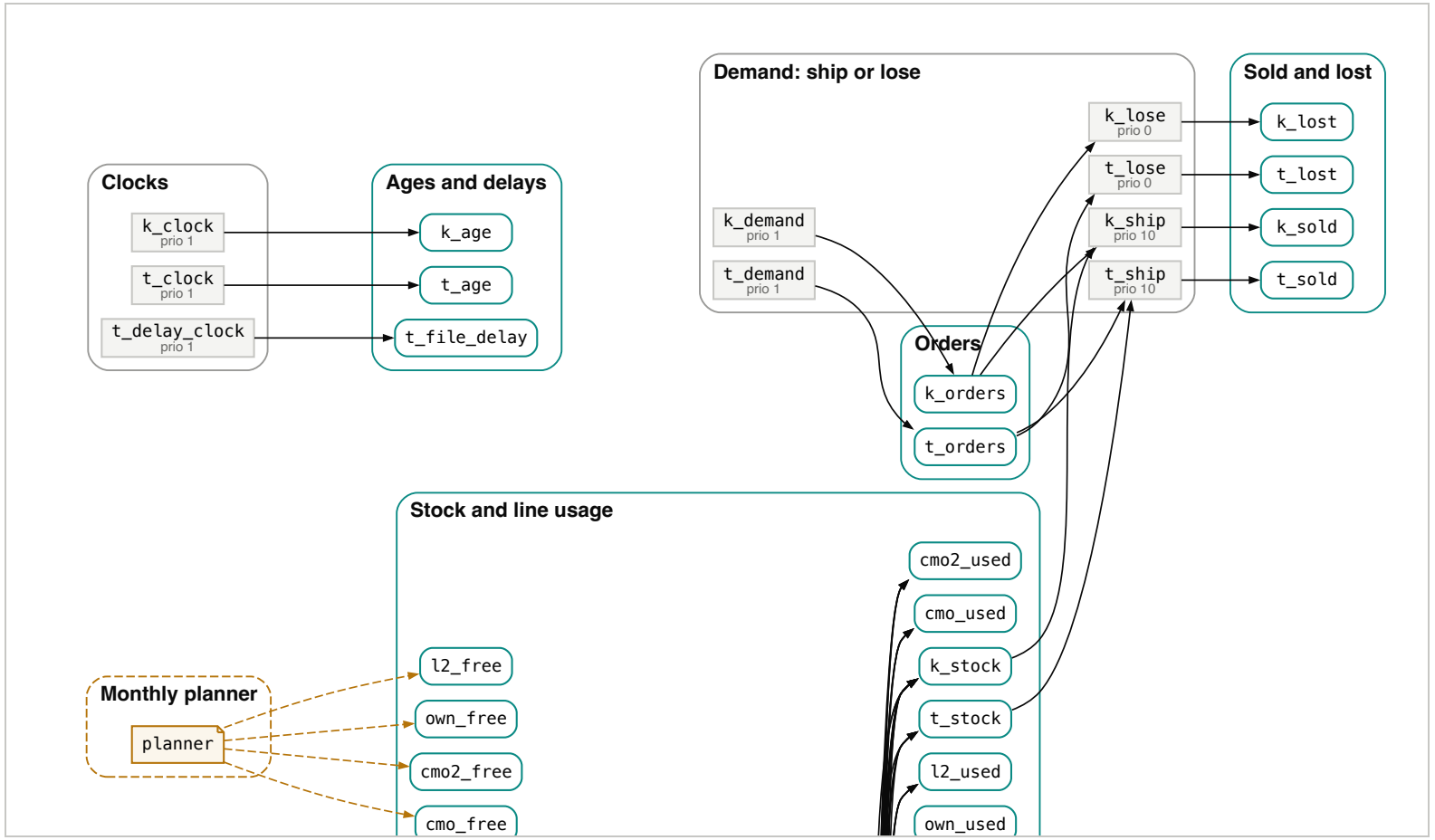
# Every sentence the operator said is an element of the model

- *“the fill site has to be in the BLA with three PPQ batches before we file”* → rule `t_file`: Tern files only once a site has 30 tenths of PPQ.
- *“I'd move Tern's PPQ in-house … and give the CMO notice”* → rule `cmo_exit_k` and one switch, `exit_at_k = 1`: the only line that differs from the rigid reservation.
- The map is generated from the document: all 48 places, 31 transitions and 27 rules, each in one box.



# The same model, every element

- **Places** (teal outline) are the stocks and pools: line capacity, stock, orders, cash.
- **Transitions** (grey) do the work, with their success chance and priority: `k_base_own` p 0.97 · prio 100.
- A **teal arc** is capacity returned when the batch is done; a black arc is consumed or produced; the number is how much.
- **Rules** (amber) are the calendar and the decisions, dashed where they set a place.
- All 48 places, 31 transitions and 27 rules. Nothing is summarized away.



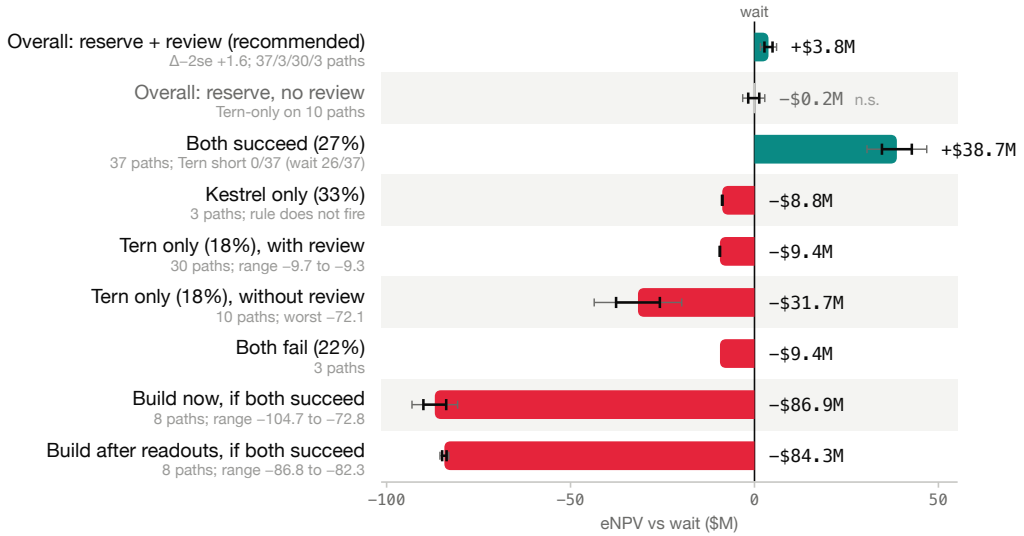

Scroll the diagram. Drawn from the engine's own view of the net (`network_graph`), inside the boxes of the map on the previous slide.

# Reserve now, review at Kestrel's readout, don't build

- **+$3.8M ± $1.1M** against waiting, at the portfolio committee's odds; +$25M at peak-year batch value.
- **At the operator's own 50% for Kestrel: break-even** (+$1.6M ± $0.9M). This is supply protection, not a bet on the odds.
- **+$39M if both succeed** (27%): Tern short at launch on 26 of 37 paths under wait, 0 of 37 reserved.
- **A bounded $9–11M cost** in every other outcome.
- **Building is $73–105M worse** even when both succeed.
- **Tern-only is safe only with a second CMO contract in writing** (–$72.5M without, 3 paths, indicative).

**Reserve the CMO now, review at Kestrel's readout: eNPV against waiting**

Paired single paths (same seed, same dates in both options), whiskers ±1 se (dark) and ±2 se (grey) across paths. Top two bars: odds-weighted overall. Others: given the outcome. Kestrel 60%, Tern 45%, 9% WACC, year-1 value per batch.



# The study ships as a generated report, not a folder

**Six parts, one template, every study:**

1. The answer and the decision rule
2. The evidence, sensitivities and caveats
3. Why a spreadsheet cannot produce it
4. The model: elements in the operator's words, and the diff that *is* the option
5. The execution map
6. How we got it: commands, commit, the seeds

**Generated, never edited.** `render_report.jl` builds it from the study's own files, so it cannot drift from them. Its prose is the study's own FINDINGS, written by the model; a critic that audits prose against numbers is W9.

**Self-contained.** One HTML file with a PDF beside it; every figure embedded, nothing to open in the repository.

**Checkable.** It names the commit, hashes both model documents, and lists all 208 stored runs by document and seed: any number can be replayed.

`agent/studies/launch_supply/REPORT.pdf`, 18 pages.

THE MOMENT WORTH WATCHING

# The model made the tail visible, and the owner supplied the option

CLAUDE

In the other outcomes the reservation costs money: Tern only, 0.18 chance, **–$18.6M to –$72.1M**. […] That is why the expected value nets to about zero.

OPERATOR

Kestrel reads out in March 2027, nine months before Tern. […] I'd move Tern's PPQ in-house, file Tern on our own line, and give the CMO notice. That means eating the $5–10M. So the reservation isn't a fixed bet on Tern; it's something we decide to keep or drop at Kestrel's readout.

CLAUDE

**Your guess holds: under the review rule, the Tern-only outcome costs $9.4M against waiting, and the $72M tail is gone.**

```
python3 exit_review.py → weighted delta +3.81, se 1.10 (30 Tern-only paths)
```

HONEST ABOUT IT

# What we are not claiming

- **The case is simulated, AI on both sides.** The operator was cooperative. It demonstrates the workflow, not anyone's supply plan.
- **Paired paths, not ensembles.** A 96-seed ensemble took 53 minutes (W24); every figure says what it rests on.
- **The interview left soft inputs untested:** demand ±30%, marketed growth and readout correlation. The published audit leads with them.
- **The prose is not yet checked.** The assistant retracted two of its own claims out loud; a critic that does this systematically is W9.
- **Contention is checked end to end; feedback and coupling are not yet** (W25).
- **The outlined pieces are the direction, not shipped.**

WHERE IT IS GOING

# Trust first, then reach

| Direction | What it changes | Tracker |
|---|---|---|
| Fast ensembles | an option-level ensemble in minutes, not 53 | W24 |
| Declared dead steps | a failed check means a bug | W20 |
| Weighting in the engine | the headline comes out of `rdx effect` | W27 |
| A critic | the briefing's prose audited against the numbers | W9 |
| Beyond contention | checks for feedback; coupled models as documents | W25 |
| Validate hook | every model edit validated automatically | W10 |
| MCP server · plugin | any assistant, no terminal; one-step install | W8 · W12 |

The first half makes the answer harder to get wrong; the second moves the copilot from something a developer sets up to something an analyst opens.